Audit Report

Own Protocol v2

Date: **July 13 – August 3, 2026**

Introduction

Own is a permissionless protocol for bringing tokenized real-world assets (RWAs) onchain. Users mint ERC-20 tokens called eTokens (e.g. eTSLA, eGOLD) using stablecoins; each eToken tracks the price of its underlying asset through onchain oracles and is backed by on-chain collateral held in Own Vaults. Where traditional RWA tokenization concentrates trust in a single custodian holding the physical asset, Own backs exposure with transparent, code-governed collateral vaults: any hedge fund, trading firm, or RWA custodian can permissionlessly operate a vault and back issued eTokens, with competition between them driving spreads.

Minting and redemption flow through a quote-driven RFQ marketplace: a buyer requests a firm off-chain quote from a market maker and either settles it atomically as a market order (`executeOrder`) or rests a limit order (`placeOrder`) for a maker to fill. Quotes are EIP-712 signed and authorized against a global signer registry, and settle inside a price band anchored to the protocol's oracle marks. Every holder also carries a code-enforced exit: a redeemer who cannot obtain a quote can force-execute against vault collateral at the oracle price after a claim window. Liquidity providers deposit collateral into ERC-4626 `OwnVault`s and share the fees and yield the vault earns; a `BorrowManager` additionally lets users borrow stablecoins against their eToken collateral.

This third review covers the protocol's in-house lending venue and its oracle migration, together with a full re-audit of the previously reviewed core and PSM contracts. The lending venue (`OwnLendingPool`, `OwnAToken`, `OwnDebtToken`, `LendingRouter`, `VaultYieldManager`) replaces the external Aave dependency with an Aave-v3-compatible pool the protocol operates itself, so vault collateral earns yield and backs borrowing without third-party venue risk. On the oracle side, `ChainlinkOracleVerifier` becomes the protocol's sole price authority: Chainlink feeds are the primary source, and band-limited in-house signed quotes fill in only while a feed is quiet. `OracleVerifier.sol` and `PythOracleVerifier.sol` were retired from use during the review and moved out of the source tree.

The review combined manual analysis of every contract in scope with three independent multi-agent automated audit passes, adversarial validation of every generated finding against source, and a dedicated review of the interest-accrual and reserve-release paths. The protocol's internal accounting was attacked directly and held: the scaled-debt identity, the RWA netting identity, PSM exposure-neutrality and round-trip rounding, and the eToken reward accumulator all survived. Every Critical and High finding was verified directly against source, and every fix carries a regression test verified to fail without it. Full suite: 1,232 tests passing.

Scope

Repository: `own-protocol/own-v2`

Audited Commit: `7d5a652a17bc6f48d0befe672120797c66362cc2`

Files:

- `src/core/AssetRegistry.sol`
- `src/core/BorrowManager.sol`
- `src/core/ChainlinkOracleVerifier.sol`
- `src/core/OwnLendingPool.sol`
- `src/core/OwnMarket.sol`
- `src/core/OwnVault.sol`
- `src/core/ProtocolRegistry.sol`
- `src/core/ReserveVault.sol`
- `src/core/VaultManager.sol`
- `src/libraries/InterestRateModel.sol`
- `src/libraries/LendingMath.sol`
- `src/periphery/LendingRouter.sol`
- `src/periphery/VaultYieldManager.sol`
- `src/periphery/WETHRouter.sol`
- `src/periphery/WstETHRouter.sol`
- `src/tokens/EToken.sol`
- `src/tokens/ETokenFactory.sol`
- `src/tokens/OwnAToken.sol`
- `src/tokens/OwnDebtToken.sol`
- `src/tokens/OwnershipNFT.sol`

Findings

Each issue has an assigned severity:

Critical/High issues are directly exploitable security vulnerabilities that can lead to loss of funds and need to be fixed.

Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.

Low/Info issues are non-exploitable or informational findings that do not pose a direct security risk to the system's integrity.

Issues marked `RESOLVED` are fixed in the codebase and carry a regression test that fails without the fix. Issues marked `ACKNOWLEDGED` are accepted by design, with the rationale and any operational mitigations documented.

Issues Found

Critical	High	Medium	Low
0	3	11	9

Issues Not Fixed and Not Acknowledged

Critical	High	Medium	Low
0	0	0	0

Summary of Findings

ID	Severity	Title	Status
A3-H-01	High	`psmFillOrder` validated the settle band against a mark it then refreshed	Fixed
A3-H-02	High	`_accrue` billed a whole elapsed window at an attacker-timed rate	Fixed
A3-H-03	High	`fulfillWithdrawal` settled another LP's shares at a caller-chosen price	Fixed

A3-L-03	**Medium**	`forceExecuteOrder` was the only settle path with no price band, and its price gate could be pointed at a stale oracle leg	Fixed
A3-M-01	**Medium**	JIT capture of accrued LP yield via permissionless `distribute` and `claimEarnedInterest`	Fixed
A3-M-02	**Medium**	Concentration cap derived only from other vaults collapses a capped vault's counted collateral to zero	Acknowledged
A3-M-03	**Medium**	Borrower draws skipped the Aave health floor that every collateral-decreasing path enforces	Fixed
A3-M-04	**Medium**	`migrateToken` desyncs every PSM wrapper's ratio-jump baseline	Acknowledged
A3-M-05	**Medium**	`releaseCollateral` ignores `Paused`, so a paused vault keeps paying redeemers while its LPs are frozen	Acknowledged
A3-M-06	**Medium**	`placeOrder` and `executeOrder` gate redeem on `isActiveAsset`, so a deactivated asset traps holders	Acknowledged
A3-M-07	**Medium**	`depositRewards` has no ex-dividend snapshot, and a fee-free PSM round-trip front-runs it	Acknowledged
A3-M-08	**Medium**	Saturated `totalAssets()` let `previewDeposit` mint a near-unbounded share count	Fixed
A3-M-09	**Medium**	`ReserveVault._releaseCollateral` omits the PSM ratio-jump guard it shares a ratio with	Acknowledged
A3-M-10	**Medium**	An EOA manager bricked every LP path, and nothing prevented one	Fixed
A3-L-01	**Low**	`utilizationBps` returned zero for a zero debt cap with live debt	Fixed
A3-L-02	**Low**	`claimEarnedInterest` and `requireVaultHealthy` share one threshold, so the revenue crank consumes the exit floor	Acknowledged
A3-L-04	**Low**	`BorrowManager._convertToCollateral` divides by an unbanded signed price	Acknowledged

A3-L-05	**Low**	ETH refund helpers pay out `address(this).balance`, not the current call's surplus	Acknowledged
A3-L-06	**Low**	`unwrapWstETH` forwarded the amount Lido reported, not the amount received	Fixed
A3-L-07	**Low**	`LendingRouter` exit legs forwarded the reported aToken amount, not the amount received	Fixed
CL-L01	**Low**	A reverting aggregator bricks both price legs, including a fresh in-house price	Acknowledged
CL-L02	**Low**	Cached `clDecimals` can rot on an aggregator upgrade	Acknowledged
CL-L03	**Low**	A feed that dies mid-session reads as current for up to `clFreshWindow`	Acknowledged
CL-I01	**Info**	Compromised-signer damage cap equals `bandBps` during any quiet stretch	Acknowledged
CL-I02	**Info**	Timestamp-ignoring consumers accept prices up to `maxAnchorAge` old	Acknowledged
CL-I03	**Info**	`verifyPriceForSession` self-call drops `msg.value`	Acknowledged
CL-I04	**Info**	Multicall combined with a payable `verifyPrice`	Acknowledged

Issue A3-H-01: `psmFillOrder` validated the settle band against a mark it then refreshed RESOLVED

Severity: **High**

Vulnerability Detail

`psmFillOrder` called `_checkSettleBand` immediately before `_psmContext`, and `_psmContext` re-reads the asset mark via `pullAssetPrice` and derives the conversion ratio that the fill actually settles at. The band therefore bounded the pre-refresh mark while `_psmFillMint`, `_psmFillRedeem` and `_psmSpreadFee` all priced off the post-refresh one, so a filler's real edge was `settleBandBps` plus whatever the price drifted within `maxMarkAge`, not `settleBandBps` alone. With the Robinhood parameters (`settleBandBps` 500, `maxMarkAge` 1h): a mark of $400 written 55 minutes ago with the market at $368, and a resting Mint order at a $420 limit price – exactly the permitted edge at placement – passes the band against $400, `_psmContext` then refreshes to $368, and the filler

delivers 100 wrapper units worth $36,800 while collecting the full $42,000 escrow, an edge of 12.4% against a 5% cap. The redeem leg is symmetric at roughly 13.7% after a rise, and `psmFillSpreadShareBps` defaults to 0 so no fee offsets it.

Impact

The order owner is drained of the entire intra-`maxMarkAge` price drift on top of the intended settle band, defeating the per-unit damage cap that the band exists to provide on the PSM fill path.

Resolution

`_checkSettleBand` now runs after `_psmContext`, so the band bounds the same mark the fill settles against. `_checkSettleBand` is `view` and `_psmContext`'s `notePsmRatio` write rolls back if the band then reverts, so the only behavioural change is revert ordering. The change is localised: `psmMint` and `psmRedeem` call `_psmContext` but never `_checkSettleBand`, pricing off the oracle ratio directly, and the `_settleMint` and `_settleRedeem` call sites are quote-driven and do not refresh the mark mid-call. A regression test pins a limit sitting exactly on the pre-refresh edge and asserts the fill reverts `PriceOutOfBand` after an 8% drop, verified to fail against the pre-fix ordering.

Issue A3-H-02: `_accrue` billed a whole elapsed window at an attacker-timed rate RESOLVED

Severity: **High**

Vulnerability Detail

`_accrue` computed `accrueIndex(_index, _currentRateBps(), dt)` — the instantaneous rate sampled at call time, applied retroactively across the whole elapsed interval — and both inputs were attacker-controlled, since `accrue()` carries no modifier and the rate's denominator `maxDebtUSD()` derives from `collateralMark`, which tracks the vault's `totalAssets()`. Every path changing the numerator already accrued first; no path changing the denominator did. With rate params base 100, optimal 8000, slope1 400, slope2 7500, a $20M mark and a $5M book (350 bps premium), settling a matured $12M withdrawal through the then-ungated `fulfillWithdrawal` dropped the cap to $4M, clamped utilisation to 10000 bps and lifted the premium to 8000 bps; calling `accrue()` in the same transaction with `dt` of 7 days grew the index by 1.534% instead of 0.067%, roughly $73.4k of debt in one block. The mirror direction pins the premium at its 100 bps floor and starves LPs. The 2026-07-31 remedy hooked accrual into `OwnVault`'s six `totalAssets()` movers on a sufficiency argument that a follow-up pass refuted: `haltVault`/`unhalt` assign the mark directly with no `totalAssets()` movement, `VaultManager.pullCollateralPrice` moves it through an independent live price factor, and this pass's own A3-M-01 fix turned `VaultYieldManager._distribute` into a bare aToken transfer that raises `totalAssets()` with no vault code running.

Impact

Debt inflated by more than a percent in a single block, making every position in a band just above HF 1.0 liquidatable at `liquidationBonusBps`; run in the mirror direction, the premium is pinned at its floor and LP revenue is suppressed.

Resolution

`BorrowManager` gained `_lastPremiumBps` (appended, slot 20), and `_accrue` now bills each elapsed window at the premium observed when that window opened, then observes for the next — so no denominator move can reprice elapsed time, independently of how the mark moved, closing all three sites at once rather than chasing entry points. Only the premium is frozen; the Aave leg reads live, because its rate is external and unsteerable while the premium's denominator is caller-movable, and freezing both was implemented and reverted after a test caught a roughly 10% revenue leak to LPs. `_projectedIndex` and `_accrue` both route through `_windowRateBps()` so `debtOf`, `healthFactor` and liquidation previews agree with the transition they preview. As belt and

braces, `haltVault` and `unhalt` also book accrual before their VaultManager hook. Two residuals are recorded: `_lastPremiumBps` reads 0 on an existing proxy so the first window after upgrade falls back to a live read, and the ADMIN-only `setTargetLtvBps` still has no accrual hook, harmless under the root fix. Storage layout was re-snapshotted and verified append-only, and the regression test was verified to fail without the freeze (13,945 versus 10,050 USD).

Issue A3-H-03: `fulfillWithdrawal` settled another LP's shares at a caller-chosen price RESOLVED

Severity: **High**

Vulnerability Detail

`fulfillWithdrawal` checked only that the request exists and is `Pending` — there was no caller gate, no `assets == 0` revert, and no `minAssetsOut`, unlike `deposit` and `requestDeposit`, which both carry slippage floors. Because `SafeERC20.safeTransfer(owner, 0)` does not revert, a zero settlement succeeded silently: the owner's escrowed shares were burned, the request was marked `Fulfilled`, and 0 assets were paid — destroying a claim whose zero valuation is an accounting artifact, since `totalAssets()` saturates to 0 whenever the aToken balance drops to or below the pending-deposit escrow and recovers as the balance rebases back. The depressed-price variant is the same mechanism at a transient trough: a rival LP settles a victim's request cheaply and the difference accrues pro-rata to the remaining LPs, so an attacker holding 50% who burns a victim's 10% moves to 55.6% of the restored pool.

Impact

A third party can permanently destroy an LP's entire withdrawal claim for a zero payout, or settle it at a transient trough and capture the difference through their own share of the vault.

Resolution

`fulfillWithdrawal` now reverts `ZeroAmount` when `convertToAssets(req.shares) == 0`, placed before the halted-branch fork — the halted emergency-exit branch skips both the wait period and the utilisation gate and was the only path that actually settled at zero, since on the Active path `withdrawalBreachesUtil` reverted first. The guard is costless when the zero is genuine and saves the claim when it is an artifact. Permissionless fulfilment is kept by design, with no caller gate and no `minAssetsOut`: keeping fulfilment open preserves the option to automate settlement on users' behalf, the trough leg requires a transient dip whose every mechanism routes through preconditions absent on the deployed venue, and with `withdrawalWaitPeriod` at 0 the owner sees the price and can settle in the same block they request. Both decisions must be revisited if the vault is ever deployed against canonical Aave V3, where external liquidations make the trough manufacturable.

Issue A3-L-03: `forceExecuteOrder` was the only settle path with no price band, and its price gate could be pointed at a stale oracle leg RESOLVED

Severity: **Medium**

Vulnerability Detail

`forceExecuteOrder` applies neither `_checkSettleBand` nor `_checkPriceBand`, though both exist expressly to cap leaked-signer damage, and `placeOrder` bounds `limitPrice` only as non-zero; the original acceptance rested on the `currentPrice >= limitPrice` gate capping the payout at market. That premise is false, because `currentPrice` need not be the market: in `ChainlinkOracleVerifier.verifyPrice` the in-house branch sits behind `if (priceData.length > 0)`, so an empty proof skips a fresher cached quote and falls through to the stale Chainlink anchor, returned stamped `block.timestamp`, while `getPrice` and therefore the VaultManager mark always preferred the fresher leg — the two reads disagreed on the same asset in the same block and the caller chose which one the gate saw. This is reachable in the ordinary `(clSilence, clFreshWindow]` window, 15 minutes to 4 hours on the deployed configuration, with no key compromise: at a $368 mark against a $400 stale anchor, a 2,500-unit force-execute releases $1,000,000 of collateral to close $920,000 of exposure. A third mechanism at the same function is that it is the only fill path with no `order.expiry` check.

Impact

$80,000 of collateral released above the protocol's own valuation per execution, repeatable at LPs' expense for as long as the two oracle legs diverge.

Resolution

`verifyPrice` now takes the cached in-house price when it is fresher than the anchor, mirroring `getPrice`'s selection, before falling back within `clFreshWindow` — the caller still chooses whether to prove, but never which leg wins. It was deployed by swapping `INHOUSE_ORACLE` in the registry, an atomic change with a documented rollback and no other contract modification. The band itself was deliberately not added: `_checkSettleBand` is symmetric, so it would reject a limit price far below the mark, which is a cheap exit favouring LPs, and it reverts `StaleSettleMark`, which would break the unblockable-exit guarantee that `closeExposure`'s stale tolerance exists to preserve; a one-sided clamp would change settle semantics and is a product decision. The `order.expiry` mechanism remains open but is now genuinely benign, since the payout can no longer exceed the protocol's own valuation. Tests assert that the fresher quote wins and that `verifyPrice` and `getPrice` agree on both price and timestamp, verified to fail with the preference removed ($380 versus $364), plus a companion test pinning that the preference is by freshness and not by leg.

Issue A3-M-01: JIT capture of accrued LP yield via permissionless `distribute` and `claimEarnedInterest` RESOLVED

Severity: **Medium**

Vulnerability Detail

`VaultYieldManager.distribute` was permissionless and pushed the entire held balance through `OwnVault.shareYield` in one step, raising the share price instantly with no vesting. `claimEarnedInterest` was likewise permissionless, letting a caller first force-realize borrowers' accrued premium into the shell and then pay it to themselves. Whoever held shares at that instant took a pro-rata slice of everything accrued since the last distribution, regardless of how long they had been invested. The finding was initially rated High on the assumption that `withdrawalWaitPeriod` sat at its `0` default, making the attack atomic and flash-loanable; `broadcast/` records an 8-hour delay set against the live vault on 2026-07-20, which is why it settles at Medium.

Impact

A newcomer depositing immediately before a crank captures a pro-rata share of yield accrued entirely before they entered, diluting the incumbent LPs who earned it.

Resolution

`distribute` no longer calls `IOwnVault.shareYield`; it transfers the converted aTokens straight to the vault, which lifts the share price identically because `totalAssets()` is `balanceOf(vault) - _pendingDepositAssets`, and which removes the shared-reentrancy-guard obstacle that made a vault-side hook impossible. `VaultYieldManager.syncYield()` then performs a best-effort claim followed by a non-reverting distribute, and `OwnVault._syncLending()` calls it, after `accrue()`, on the four paths that price LP shares; `releaseCollateral` and `shareYield` stay accrue-only as they are not pricing points. `syncYield` deliberately carries no reentrancy guard — the vault calls it from inside its own guarded paths, the body is idempotent, and nothing in it calls back into the vault — and its claim leg is `try`/`catch` so a draw that would breach the vault's Aave health floor never blocks LP flow. Two operational decisions ride with the fix: `interestBufferBps` moves to 100 bps as the `BorrowManager` constructor default, with deployed managers needing `setInterestBufferBps(100)`, and `withdrawalWaitPeriod` moves to 0. The regression test sweeps revenue into the shell, has an attacker deposit nine times the incumbent's stake, and asserts the attacker redeems only their principal; verified to fail with the hook removed.

Issue A3-M-02: Concentration cap derived only from other vaults collapses a capped vault's counted collateral to zero ACKNOWLEDGED

Severity: **Medium**

Vulnerability Detail

`_cappedContribution` derives a capped vault's allowance purely from other vaults' counted collateral, as `others·cap/(BPS−cap)`, so it floors to zero when the capped vault is the only counted vault and collapses super-linearly as the rest of the pool shrinks — vault A holding $9M raw and counting $428,571 drops to $428 once uncapped vault B drains from $1M to $1,000, reverting all `openExposure` with `CollateralNotInitialized` and all `fulfillWithdrawal` with `MaxUtilizationExceeded`. A second mechanism was found while modelling the fix: `withdrawalBreachesUtil` subtracts only the withdrawn amount from the current `_globalCollateralUSD` and never recomputes the capped vault's contribution, which is itself a function of the balance being withdrawn, so a withdrawal passes the gate and is then pushed over the cap retroactively by the next permissionless `pullCollateralPrice` — at cap 3000 and util 5000 with both vaults at $5M, a $3.14M withdrawal clears at exactly 50% and utilisation then settles at 75.4% with nothing reverting.

Impact

Availability only, with no fund loss: mints and all further withdrawals freeze while real collateral is counted at a fraction of its value, with no in-contract recovery beyond disabling the cap.

Resolution

Not fixed, because `VaultManager` is immutable in the live system — only `OwnVault`, `BorrowManager` and `VaultYieldManager` are redeployable — so a code change could never reach the deployed instance; a self-referential floor was implemented and reverted on that basis, and it would in any case address the first mechanism only. Live exposure is nil: no `setCollateralCapBps` call exists anywhere in `broadcast/`, so every deployed cap is 0 and the branch is unreachable, and leaving every cap at 0 is the mitigation since a non-zero cap is the only thing that arms it. This is recorded as a standing ops rule not to set a non-zero concentration cap on the live `VaultManager`, with `setAssetCapUSD` or a deposit cap as the alternative concentration control; recovery if one is ever set is `setCollateralCapBps(vault, 0)`, and any future `VaultManager` redeploy must fix both mechanisms before caps are safe to enable.

Issue A3-M-03: Borrower draws skipped the Aave health floor that every collateral-decreasing path enforces RESOLVED

Severity: **Medium**

Vulnerability Detail

`_drawFromAave` had two call sites with asymmetric treatment: `claimEarnedInterest` drew and then reverted below `minClaimHealthFactor`, while `_executeBorrow` drew with no check at all, even though every collateral-decreasing path – `fulfillWithdrawal` and `releaseCollateral` – enforces that same floor via `requireVaultHealthy()`. Any `targetLtvBps` greater than the venue's liquidation threshold divided by 1.1 therefore reached a health factor in the interval between 1.0 and the floor at the protocol's own debt cap, and ordinary Aave interest accrual could walk the vault into that band with no attacker involved at all.

Impact

A band in which LP exits and collateral releases revert while borrowing keeps succeeding, freezing LPs out of the vault while the debt that trapped them can still grow.

Resolution

After `_drawFromAave(stablecoinAmount)` in `_executeBorrow`, the vault's Aave health factor is read and the call reverts `VaultUnsafeHealthFactor` below `minClaimHealthFactor` – the same post-draw pattern `claimEarnedInterest` already used. A borrow can now never create the frozen-exit band; at worst it reverts at the floor, which is the correct side to block. Set-time validation of `targetLtvBps` against the venue's liquidation threshold was deliberately not added, since the runtime check subsumes it and the venue threshold is not uniformly exposed at set time, though confirming the deployed value remains on the ops checklist as defence in depth. This is a direct sequel to H-07, which added `requireVaultHealthy()` to the collateral-decreasing paths but never to the debt-increasing one. The regression test asserts a borrow at mock HF 1.05 reverts while one at exactly the 1.1 floor succeeds, verified to fail against the pre-fix code.

Issue A3-M-04: `migrateToken` desyncs every PSM wrapper's ratio-jump baseline ACKNOWLEDGED

Severity: **Medium**

Vulnerability Detail

`migrateToken` rescales `_legacyRatio` and the asset mark, via `applySplit`, but does not touch `_psmConfigs[ticker][*].lastUsedRatio`. After a split the derived PSM ratio, computed as `wrapperPrice·PRECISION/mark`, therefore shifts by exactly the split ratio, and the jump guard reads that shift as a discontinuity.

Impact

Every PSM path for the migrated ticker — mint, redeem and fill — reverts `RatioJumpExceeded` until an operator re-arms the guard.

Resolution

No code change: a token migration is an operator-scheduled event, so the baseline update belongs in the migration runbook rather than in the contract. The runbook announces the migration and a PSM hold, calls `setPsmPaused` for every wrapper of the ticker, runs `migrateToken`, calls `resetRatioGuard` per wrapper, performs one controlled PSM operation per wrapper, and then unpauses. One residual is accepted knowingly: `resetRatioGuard` re-arms by zeroing the baseline, so the first PSM operation per wrapper after each reset runs with the jump guard disarmed, which the controlled first operation under an operator's own supervision is what covers. A second mechanism — legacy-ratio decay to zero after roughly nineteen successive 1-for-10 reverse splits — is folded in here and noted with no action, as it is not credible standalone.

Issue A3-M-05: `releaseCollateral` ignores `Paused`, so a paused vault keeps paying redeemers while its LPs are frozen ACKNOWLEDGED

Severity: **Medium**

Vulnerability Detail

Vault `pause()` stops LP entry and exit, but `releaseCollateral` carries no vault-status check, so collateral keeps leaving a paused vault through eToken redemption and force-execution while the vault's own LPs cannot withdraw. The finding read `pause()` as a full freeze and treated the redeemer path as an unintended bypass of it.

Impact

During an incident pause, collateral continues to leave the vault through the redemption path while LPs are held in place.

Resolution

By design — `pause()` is an LP pause, not a full freeze. Its purpose is to stop LP entry and exit, for example the exit-ahead-of-bad-debt window tracked as M-13, while eToken redemption, which is the protocol's unblockable-exit guarantee, keeps flowing. Redemption at a correct mark is approximately value-neutral for the vault, since collateral leaves as matching exposure closes, so it is not the drain the pause exists to contain. The finding's force-execute premise was also wrong: `forceExecuteOrder` has its own independent levers, reverting `AssetPaused` under the global or per-asset trading pause — both instant operator actions — and the per-asset force-execute vault allowlist can drop any vault as a collateral source, while `releaseCollateralForBadDebt` is reachable only through the trusted-operator `absorbBadDebt` flow. M-13's acceptance is unaffected, since pause still freezes the LP exits M-13 cares about.

Issue A3-M-06: `placeOrder` and `executeOrder` gate redeem on `isActiveAsset`, so a deactivated asset traps holders ACKNOWLEDGED

Severity: **Medium**

Vulnerability Detail

`placeOrder` and `executeOrder` apply `_validateAsset` unconditionally rather than on the Mint branch only, so `setAssetActive(ticker, false)` blocks redeem order entry as well as mint entry. A holder of the deactivated asset therefore cannot open a self-serve exit for as long as the deactivation stands.

Impact

For a ticker with no funded PSM wrapper, holders' exit timing during a deactivation depends entirely on an operator action.

Resolution

By design — `setAssetActive(ticker, false)` is meant to freeze new order flow for the ticker entirely, not to open a self-serve wind-down, and the Mint-only gate on the fill paths added under L-17 exists so already-resting orders can complete, not as a template for order entry. Holder exits during a deactivation are operator-managed and every remedy is an instant operator action: reactivate the asset, run the halt path so `redeemHalted` becomes available, or keep a PSM wrapper redeemable. The residual exit-timing dependency is accepted as consistent with `setAssetActive` being a trusted-operator lever.

Issue A3-M-07: `depositRewards` has no ex-dividend snapshot, and a fee-free PSM round-trip front-runs it ACKNOWLEDGED

Severity: **Medium**

Vulnerability Detail

`EToken.depositRewards` credits the reward accumulator against the supply as it stands at deposit time, with no ex-dividend snapshot, so an entrant who acquires eTokens immediately before the deposit and exits immediately after collects a pro-rata slice of a dividend that accrued entirely before they held. The PSM round trip that would fund such a position carries no fee, so the entry and exit are close to costless.

Impact

Would be a direct transfer from long-term holders to a just-in-time entrant, at roughly High severity, if the distribution channel were live.

Resolution

Dormant — the dividend channel is verified not live. Robinhood Gen-2 stock tokens pay no on-chain cash dividends; dividends auto-reinvest into the wrapper's `uiMultiplier` and therefore surface in this system as PSM-ratio drift and reserve surplus, skimmed by the market maker or operator through the existing surplus machinery. `depositRewards` appears in no deploy or ops script, so with no distribution pot there is nothing to front-run. A precondition is recorded: if surplus-to-eToken-holder distribution ever goes live, the capture becomes real and a mitigation must ship first — a pre-announcement balance snapshot, a streamed or dripped payout, or the mint-below-wrapper-value design in which entrants receive eTokens priced at the bare share mark while paying the multiplier-inclusive wrapper, which is the direction chosen if this is ever built.

Issue A3-M-08: Saturated `totalAssets()` let `previewDeposit` mint a near-unbounded share count RESOLVED

Severity: **Medium**

Vulnerability Detail

`totalAssets()` saturates to 0 when the aToken balance drops to or below the pending-deposit escrow, which is the H-06 underflow fix's own consequence. With shares outstanding and `totalAssets()` at 0, OpenZeppelin's `previewDeposit` denominates against the virtual `+1`, so on the live oUSDG vault — a 6-decimal asset with `_decimalsOffset() = 6` — a single `deposit(1000)` costing 0.001 USDG against a 1e18 share supply mints roughly 1e21 shares, about 99.9% of the vault; the decimals offset defends the empty-vault case, not zero-assets-with-live-supply. The 2026-07-31 fix guarded only the manager-gated `acceptDeposit`, while `deposit()` (both overloads, via `_depositWithMin`) and `mint()` price through the same saturating `totalAssets()` and `_requireDepositApproval` is `false` by default, so the permissionless path was left open. Both release paths bound with an inclusive `if (amount > totalAssets()) revert`, so a release of exactly `totalAssets()` lands on zero in one call, and the escrow variant is sharper still, since releasing exactly the backing leaves the escrow as the entire raw balance and `requireVaultHealthy()` — which reads the venue's raw balance rather than `totalAssets()` — still passes. `acceptDeposit` was additionally the only share-minting path that never read `_vaultStatus`.

Impact

Prior LPs diluted to a rounding error, permanently — `_mint` records no cost basis and `OwnVault` has no admin burn, so shares bought for dust keep their proportion of every later inflow long after the zero-assets state heals.

Resolution

The check moved into a private `_requireSolvent()` invoked by `_depositWithMin`, `mint` and `acceptDeposit` — one definition, three call sites — placed after `_syncLending()` in each, matching the original placement, so realized yield can lift a transiently saturated balance rather than the guard rejecting a vault the sync would have rescued; `acceptDeposit` also gained the `whenDepositsAllowed` modifier its siblings already carried. Tightening the two release bounds from `>` to `>=` was considered and not bundled, since it blocks the state rather than its exploitation and would make a force-execution against a vault's full backing start reverting, which is a product decision. The deployed bytecode on the live vault carries no guard on any of the three paths, but the precondition is not attacker-reachable as configured — the venue is the in-house `OwnLendingPool` with no liquidation function and a flat 1:1 `OwnAToken`, and `forceExecuteOrder`

reverts `ForceNotEnabled` while `claimThreshold` is 0 — which is a configuration property rather than a code property, so this fix must ship before force-execution is enabled and before any vault is pointed at canonical Aave V3. Six tests cover the attack paths and the over-blocking cases, with the four attack tests verified to fail with the guard removed.

Issue A3-M-09: `ReserveVault._releaseCollateral` omits the PSM ratio-jump guard it shares a ratio with ACKNOWLEDGED

Severity: **Medium**

Vulnerability Detail

The surplus clamp in `_releaseCollateral` values the reserve at the raw `wrapperPrice/mark` ratio with no `ratioJumpBoundBps` check, so a wrapper feed printing 10% high makes phantom surplus skimmable — while the identical print reverts `RatioJumpExceeded` on every `OwnMarket` PSM path that consumes the same ratio.

Impact

A semi-trusted caller could withdraw reserve backing as phantom surplus during an oracle misprint.

Resolution

Acknowledged, not fixed — the reserve vaults are live on Robinhood Chain and are not upgradable, so a code guard could only ever reach future deployments. The trigger requires a semi-trusted caller, either an allowlisted maker `withdraw` or a manager-operator `skimExcess`, together with a simultaneous oracle misprint, and after the Chainlink migration the wrapper feed is Chainlink-primary with in-house proofs bounded to the anchor band, so the reachable misprint is small. Ops mitigation: sanity-check the wrapper feed against the PSM's `lastUsedRatio` before any skim or maker withdrawal, and include the `_psmContext`-mirror guard in any future `ReserveVault` deployment.

Issue A3-M-10: An EOA manager bricked every LP path, and nothing prevented one RESOLVED

Severity: **Medium**

Vulnerability Detail

`_syncLending` calls `try IVaultYieldManager(manager).syncYield() {} catch {}`, but for a void-returning external call solc emits an extcodesize check that reverts in `OwnVault`'s own frame, outside the `try`, so it cannot be caught. An EOA manager therefore reverts `deposit`, `mint`, `acceptDeposit` and `fulfillWithdrawal`; `fulfillWithdrawal` is the only exit, since `withdraw` and `redeem` hard-revert and `maxWithdraw`/`maxRedeem` return 0, while `requestWithdrawal` does not call `_syncLending`, so shares could still be escrowed into a queue that can never drain. Neither write site checked: the constructor and `setManager` validated only the zero address, while `manager`'s own NatSpec asserted that it is always a contract and never an EOA, and `VaultYieldManager`'s contract docs actively instructed operators into the failure by describing uninstallation as setting the manager back to an EOA. The invariant was asserted in three places and enforced in none.

Impact

A single admin `setManager` call to an EOA freezes every LP entry and exit, with already-escrowed withdrawal shares trapped in a queue that cannot drain.

Resolution

`ManagerNotContract` now reverts on `code.length == 0` in both the constructor and `setManager`. Enforcing at the boundary is what lets `_syncLending` keep calling the manager bare rather than re-checking on every LP action, and `VaultYieldManager`'s uninstall note now names a no-op shell contract instead of an EOA. This guarantees only that the manager has code, not that it is correct: a contract without `syncYield` still reverts and is absorbed by the `try`/`catch` by design, which is pinned by test so the fix cannot convert a caught failure into a hard one. There is a deployment consequence — `CreateVault.s.sol`, `DeployRobinhood.s.sol` and `DeployMainnet.s.sol` all pass an address straight to the constructor, and historically that address was an operator EOA, so `VM_ADDRESS` and `vm_` must be a contract from this change onward; the live vault is unaffected, as its manager was already set to the `VaultYieldManager`.

Issue A3-L-01: `utilizationBps` returned zero for a zero debt cap with live debt RESOLVED

Severity: **Low**

Vulnerability Detail

`utilizationBps`'s `if (cap == 0) return 0` assumed that a zero cap implies zero debt, but `onVaultHalted` zeroes `_collateralMark` while `_totalScaledDebt` is untouched. Halting a fully-drawn vault therefore cut the premium from 6125 bps to the 100 bps floor at the exact moment LPs exit unconditionally, and rewarded borrowers for not repaying during a halt. The same branch is reachable at genesis before the first `pullCollateralPrice`, and whenever `onCollateralReleased` drives the mark to zero.

Impact

Forgone premium for LPs precisely during a halt; `_flooredIndex` pins book debt at or above pool debt, so LPs carried no shortfall risk.

Resolution

The branch now reads `if (cap == 0) return _totalScaledDebt == 0 ? 0 : BPS;`, so a zero cap with live debt registers as full utilisation. Two tests cover it: the full-utilisation case, verified to fail pre-fix, and a companion pinning the genuinely-idle branch that the original `return 0` existed for.

Issue A3-L-02: `claimEarnedInterest` and `requireVaultHealthy` share one threshold, so the revenue crank consumes the exit floor ACKNOWLEDGED

Severity: **Low**

Vulnerability Detail

`claimEarnedInterest` gates on the vault's Aave health factor being at or above `minClaimHealthFactor`, and `requireVaultHealthy()` reads the identical storage variable, so a junior revenue claim can in principle consume margin that the senior claims — LP exit and force-redemption — depend on, converging on the floor because the health factor is re-read after the draw. The A3-M-01 fix makes this materially more reachable, since the claim now fires on every LP entry and exit rather than on an occasional crank.

Impact

Repeated claims can walk the vault's Aave health factor down toward the same value LP exits are gated on, leaving exits possible only at equality.

Resolution

Accepted with no code change. Claimed interest is the premium spread on outstanding debt, orders of magnitude smaller than the LP withdrawal flows it would have to crowd out, so the margin it can consume is immaterial, and a dedicated buffer parameter — storage, setter and ops surface — costs more than it protects; a buffered variant was implemented and reverted on that basis. It is self-limiting in any case: convergence is geometric, `distribute()` returns most of the drawn amount to the vault as collateral, and a breaching claim is skipped by the `try`/`catch` rather than blocking the LP's transaction. Gating the claim on `minClaimHealthFactor` plus a buffer is the recommended follow-up if claim sizes ever approach withdrawal scale.

Issue A3-L-04: `BorrowManager._convertToCollateral` divides by an unbanded signed price ACKNOWLEDGED

Severity: **Low**

Vulnerability Detail

`_convertToCollateral` verifies the collateral price for staleness only and then uses it directly as a divisor, skipping the `_checkPriceBand` that guards `_executeBorrow` and `liquidate`. Since `absorbBadDebt`'s `absorbAmount` is operator-chosen, the resulting release is an operator input multiplied by an unbanded price, bounded only by `totalAssets()`.

Impact

Under a mispriced or leaked-signer collateral proof, bad-debt absorption could release more collateral than the debt warrants, bounded by the vault's backing.

Resolution

Accepted — the path is operator-gated and the destination is the fixed registry treasury, which bounds where value can land. The band may be structurally unavailable here, since `_checkPriceBand` reads `vmgr.assetMark(collatAsset)` and a collateral-only ticker may legitimately carry no mark, so the practical form would be a clamp against the cached collateral mark, which buys little over the trust assumption already required to reach the function. Related to L-16, which accepts over-socialization through the same function's `absorbAmount`. Severity was downgraded from Medium to Low on the retirement of `OracleVerifier`, since in-house proofs are now anchor-band bounded.

Issue A3-L-05: ETH refund helpers pay out `address(this).balance`, not the current call's surplus ACKNOWLEDGED

Severity: **Low**

Vulnerability Detail

Both ETH refund helpers forward `address(this).balance` on the stated premise that the contract has no `receive`, so its balance can only be the current call's surplus. That premise is false: SELFDESTRUCT and coinbase payments bypass `receive`, so force-fed ETH is swept by the next payable caller, and a single wei bricks `liquidate` for a contract caller lacking a payable `receive`.

Impact

Force-fed ETH can be swept by an unrelated caller, and one wei is enough to grief `liquidate` for contract callers.

Resolution

Fix the comment, not the code. No path on the deployed venue forwards ETH — `verifyFee` is always 0 for `ChainlinkOracleVerifier` and Robinhood Chain has no oracle fee leg — so the sweep has nothing to sweep and the griefing variant has no victim. The false invariant was judged the real hazard, since a future refactor could build on it, so both comments now state the actual behaviour and record the `balance - msg.value` snapshot to apply if a fee leg is ever introduced.

Issue A3-L-06: `unwrapWstETH` forwarded the amount Lido reported, not the amount received RESOLVED

Severity: **Low**

Vulnerability Detail

`unwrapWstETH` transferred the `stETHAmount` that `wstETH.unwrap` returns. Lido computes `getPooledEthByShares(w)` and then transfers `getSharesByPooledEth(stETHAmount)` shares — a second floor — so the router is credited up to 2 wei less than the figure it was handed. The router is stateless, with no stETH buffer and no rescue function, and `unwrapWstETH` is the only stETH exit, so `safeTransfer(receiver, stETHAmount)` reverts `ERC20InsufficientBalance` whenever the double floor loses a wei; only stray dust left by the deposit path's own rounding masked it, making the failure intermittent rather than clean. M-10 fixed exactly this defect on the deposit leg, where `_depositStETHInternal` already measures a balance diff and carries a comment noting that Lido delivers one to two wei short, so the two halves of one file disagreed about whether Lido's reported figure can be trusted.

Impact

Intermittent complete denial of service on the only stETH exit path, with no rescue route for the funds in flight.

Resolution

Snapshot `stETH.balanceOf(address(this))` around `wstETH.unwrap` and forward the measured delta, mirroring `_depositStETHInternal` exactly. `MockRoundingStETH` gained a `transfer` override — it previously shorted only `transferFrom`, which is why M-10's regression test never covered this half — and the new test was verified to fail against the pre-fix code with `ERC20InsufficientBalance(router, 9.999e18, 1e19)`, the exact production failure. Live exposure is none, as `WstETHRouter` is not deployed on Robinhood Chain; this matters for a future EVM deployment that reuses the periphery.

Issue A3-L-07: `LendingRouter` exit legs forwarded the reported aToken amount, not the amount received RESOLVED

Severity: **Low**

Vulnerability Detail

Both `LendingRouter` exit legs passed a reported figure to `IAaveV3Pool.withdraw` rather than the amount the router actually held: `withdrawFromVault` forwarded the `assets` that `vault.fulfillWithdrawal` returns, and `withdraw` forwarded the caller-supplied `aTokenAmount`. Under a canonical Aave pool an aToken transfer is a scaled-balance round trip — `rayDiv` in, `rayMul` out — which can credit the recipient a wei less than the sender reports, so the pool is asked to burn more than the router owns and the entire exit reverts. The caller has no workaround on the vault leg, where `assets` is derived internally from `convertToAssets(shares)` and there is no parameter to nudge down. The same file's `deposit` already measures its receipt with a balance diff around `pool.supply`, so once again the two halves of one file disagreed; this is the identical defect class as M-10 and A3-L-06, and the sweep in commit `5457dbb` touched only `WstETHRouter.sol`, so `LendingRouter` was missed rather than excluded.

Impact

Complete denial of service on both router exit legs against a canonical Aave pool, with no caller-side parameter available to work around it on the vault leg.

Resolution

Both legs now snapshot `IERC20(aToken).balanceOf(address(this))` around the inbound transfer and withdraw the measured delta, so all three legs of the file use one pattern; `pool.withdraw` returns the amount actually moved, so `underlyingAmount` and the emitted events stay truthful, and the `minAssetsOut` guard is still evaluated against the vault's reported `assets` before the withdraw and is unaffected. Passing `type(uint256).max` was considered and rejected: it is correct only under the global invariant that the router holds nothing between transactions, and it would sweep any donated aToken into the next caller's exit, so a test pins that donated aToken stays put. `MockAToken` gained a `setTransferShortfall` helper, since the strictly 1:1 mock was precisely why no existing test could reach this path, and both new tests were verified to fail against the pre-fix code with a burn-exceeds-balance revert, the exact production failure mode. Live exposure is none: `OwnAToken` is a plain 1:1 ERC-20 with no liquidity index or scaled math, and canonical Aave is wired only on the wound-down Base deployment.

Issue CL-L01: A reverting aggregator bricks both price legs, including a fresh in-house price ACKNOWLEDGED

Severity: **Low**

Vulnerability Detail

`_chainlink()` lets a revert from `latestRoundData()` bubble up. A stale feed correctly fails over to the in-house leg, but a reverting one — an unset or bricked proxy — denies service to `getPrice` and blocks `updatePrice`, taking the in-house leg down with it even when a fresh signed price is available.

Impact

Availability only; funds are safe and the state is recoverable by an admin.

Resolution

Acknowledged and won't fix. A `try`/`catch` fallback was considered and declined: with no anchor the in-house leg is unusable by design anyway, so failing closed is the intended posture. Recovery is via `setChainlinkConfig` or `disableAsset`. Note that the related `ZeroMultiplier` hard-revert in the `multiplierToken` branch is a different code path and is not covered by this acceptance; it is carried separately as a lead.

Issue CL-L02: Cached `clDecimals` can rot on an aggregator upgrade

ACKNOWLEDGED

Severity: **Low**

Vulnerability Detail

`setChainlinkConfig` caches the aggregator's `decimals()` once at configuration time. An aggregator upgrade behind the proxy that changed the decimal count would silently mis-scale every price for that asset by a factor of 10^n, with no on-chain signal that anything had changed.

Impact

Silent and severe mis-scaling of an asset's price if it ever occurs.

Resolution

Acknowledged — probability is very low, since all current Robinhood feeds are 8-decimal, while re-reading decimals on every price read would add cost to the hot path. Mitigated operationally: aggregator-upgrade monitoring across all feed proxies is a standing item on the ops checklist, and any reconfiguration through `setChainlinkConfig` re-caches the value.

Issue CL-L03: A feed that dies mid-session reads as current for up to `clFreshWindow` ACKNOWLEDGED

Severity: **Low**

Vulnerability Detail

The timestamp clamp assumes that the absence of an update implies the price is still within the feed's deviation band, which fails if feed infrastructure dies silently. A feed that has been quiet for fifteen minutes is indistinguishable on-chain from a dead one, so the assumption cannot be tested from within the contract.

Impact

A dead feed's last price is served as current for up to `clFreshWindow`.

Resolution

Acknowledged as inherent to the freshness semantics. `clFreshWindow` was reduced from 24.5 hours to 12 hours and then to 4 hours precisely to shrink this window, which is the only lever available in-contract. Mitigated operationally by feed-age alerting below the 4-hour bound on the ops checklist.

Issue CL-I01: Compromised-signer damage cap equals `bandBps` during any quiet stretch ACKNOWLEDGED

Severity: **Info**

Vulnerability Detail

The silence gate opens routinely mid-session, not only on weekends, so a compromised in-house signer can move the served price up to `bandBps` off the Chainlink anchor whenever the feed is quieter than `clSilence`.

Impact

Under signer compromise, the served price can deviate from the anchor by up to the band during any quiet stretch of the trading day.

Resolution

By design — an explicit decision that the band, not a market-hours calendar, is the security boundary. A second belt sits behind it: the `BorrowManager` and `OwnMarket` band checks against VaultManager marks, plus an instant `removeSigner`.

Issue CL-I02: Timestamp-ignoring consumers accept prices up to `maxAnchorAge` old ACKNOWLEDGED

Severity: **Info**

Vulnerability Detail

`VaultManager._resolvePrice` discards the timestamp the verifier returns, so a mark pull will accept a price up to `maxAnchorAge` old without registering it as stale.

Impact

Weekend and after-hours marks track the previous session's close rather than a live price.

Resolution

By design and intended: weekend marks track Friday's close, optionally refined by band-limited in-house quotes. One scope note is recorded — this acceptance covers mark pulls only. It does not cover `OwnMarket.forceExecuteOrder`'s collateral leg, whose own comment asserts that the price must be current; that effect is carried separately as a lead, and the settle-side half of it was fixed under A3-L-03.

Issue CL-I03: `verifyPriceForSession` self-call drops `msg.value`

ACKNOWLEDGED

Severity: **Info**

Vulnerability Detail

`verifyPriceForSession` invokes `this.verifyPrice(...)`, which forwards no ETH, so any value sent with the outer call is not passed through to the inner one.

Impact

None on the deployed configuration.

Resolution

By design and harmless — `verifyFee` is always 0 for this verifier, so there is no fee for the self-call to forward and no value can be stranded by the omission.

Issue CL-I04: Multicall combined with a payable `verifyPrice`

ACKNOWLEDGED

Severity: **Info**

Vulnerability Detail

OpenZeppelin's `Multicall` carries a known `msg.value`-reuse hazard when it is combined with payable functions, and `verifyPrice` is payable, so the pattern was flagged for review.

Impact

None — verified non-issue.

Resolution

Verified not applicable: no function in the contract reads `msg.value`, so there is no value for a batched call to reuse and the known hazard has no expression here. Left as-is.